

AI-Augmented Governance of Non-Human Identities in DevSecOps Pipelines: A Systematic Review and Risk-Scoring Framework

Manu Prakash Bhat and Merin Meleet

Abstract—Software delivery pipelines in modern enterprises rely extensively on non-human identities (NHIs), including API tokens, service principal credentials, deploy keys, and machine certificates, to authenticate automated processes across cloud-native environments. Although these machine credentials outnumber human identities by ratios often exceeding 45:1, they receive far less governance scrutiny, leaving organizations with an expanding and poorly characterized attack surface. In this paper we present a systematic review covering 127 peer-reviewed publications, industry standards, and technical reports published between 2018 and 2026. We organize contributions along five governance dimensions: discovery, risk quantification, policy enforcement, remediation automation, and continuous monitoring. Drawing on the gaps revealed by this synthesis, we introduce a composite risk-scoring framework that combines deterministic multi-factor assessment with large language model enrichment to produce normalized severity classifications across heterogeneous credential types. No prior model in the literature offers a unified scoring methodology that renders disparate credential categories comparable on a single numeric scale. We additionally propose a five-layer reference architecture whose modular design allows individual components to be deployed, upgraded, or disabled independently without disrupting the remaining pipeline. Evaluation through three organizational case studies shows that mean time to credential rotation drops by 63%, dormant identity detection improves by 78%, and policy compliance exceeds 94% within eight weeks. We close with a discussion of open challenges, among them ephemeral credential risk modeling, cross-cloud identity federation governance, and adversarial robustness of AI-augmented policy engines.

Index Terms—Non-human identity, DevSecOps, identity governance, risk scoring, machine credentials, certificate lifecycle, policy enforcement, large language models, cloud security, API token management.

I. INTRODUCTION

CONTINUOUS integration and continuous delivery pipelines have reshaped authentication practices across the software industry. Where human operators once provisioned credentials by hand and reviewed access lists on a quarterly basis, today's DevSecOps environments depend on thousands of machine-to-machine credentials that operate without any direct human oversight. Personal access tokens, OAuth application credentials, service principal secrets, deploy keys, SSL/TLS certificates, API keys, workload identity federation tokens: collectively, these non-human identities (NHIs) form the connective tissue of automated software delivery.

M. P. Bhat and M. Meleet are with the Department of Information Science and Engineering, RV College of Engineering, Bengaluru 560059, Karnataka, India (e-mail: manuaprakashb.is22@rvce.edu.in; merinmeleet@rvce.edu.in).

How large is the problem? Industry estimates place the ratio of non-human to human identities between 25:1 and 50:1 in mature cloud-native organizations [1]. A 2025 Cloud Security Alliance survey found that 68% of respondents lacked a comprehensive machine-credential inventory, while 54% had suffered at least one security incident traceable to a compromised or mismanaged NHI in the preceding eighteen months [2]. Stolen credentials accounted for 44.7% of breaches in the 2024 Verizon DBIR, with automated credentials increasingly prominent in that tally [3].

What makes NHI governance qualitatively harder than human identity management? Three factors stand out. First, human identities enjoy mature frameworks (role-based access control, multi-factor authentication, periodic reviews, behavioral analytics) that have no NHI equivalents at comparable maturity. Second, machine credentials tend to be static, long-lived, and subject to privilege creep through configuration drift. Third, the sheer heterogeneity of NHI types precludes any single governance mechanism from addressing all categories effectively.

Several converging trends sharpen the urgency. Supply chain attacks targeting CI/CD credentials have escalated rapidly, as the Codecov bash uploader compromise of 2021, the CircleCI secret exfiltration of January 2023, and continued exploitation of overprivileged GitHub Actions tokens attest [4], [5]. Regulatory pressure is mounting: the EU Cyber Resilience Act and updated NIST guidance now mandate machine identity lifecycle controls [6]. Meanwhile, the adoption of infrastructure-as-code (IaC) and GitOps has diffused secret management responsibility away from centralized security teams toward distributed development squads, fragmenting governance further [7].

Despite all this, academic coverage of NHI governance remains siloed. Certificate lifecycle work lives in PKI research. API key security appears in web application literature. Service principal governance is discussed in cloud security venues. Secret sprawl detection sits in software engineering journals. No single review has attempted to unify these threads into a governance framework for modern DevSecOps pipelines.

We address that gap with three contributions:

- 1) A systematic review of 127 publications spanning identity governance, credential lifecycle management, certificate monitoring, and policy-as-code enforcement, organized across five governance dimensions.
- 2) A composite risk-scoring framework producing normalized 0–100 severity scores across heterogeneous NHI types

via weighted multi-factor assessment, with optional LLM enrichment for contextual risk explanation.

- 3) A layered reference architecture that integrates modular discovery agents, deterministic scoring, declarative policy evaluation, AI orchestration, and multi-channel reporting into existing CI/CD workflows without sacrificing development velocity.

The paper proceeds as follows. Section II establishes background on NHI types and DevSecOps security practices. Section III describes the review methodology. Section IV presents a taxonomy of governance approaches. Section V details the proposed scoring framework. Section VI describes the reference architecture. Section VII reports evaluation results. Section VIII discusses implications and threats to validity. Section IX identifies open research challenges. Section X concludes.

II. BACKGROUND AND RELATED CONCEPTS

A. Non-Human Identities: Definition and Taxonomy

We define a non-human identity as any credential, certificate, token, or piece of cryptographic material that automated processes, services, or machines use to authenticate against other systems without real-time human involvement [8]. Several distinct categories exist, each carrying unique lifecycle characteristics and risk profiles.

Personal Access Tokens (PATs) are long-lived bearer tokens issued by platforms such as GitHub, GitLab, and Azure DevOps for API authentication. Classic PATs carry coarse-grained scopes (e.g., full repository access); fine-grained tokens restrict access to specific repositories and permission sets [9]. Their security concern is straightforward: because PATs are often stored in CI/CD environment variables, any workflow step can read them.

Service Principal Credentials allow applications in cloud environments (Azure AD, AWS IAM, GCP) to authenticate against cloud APIs through client secrets or X.509 certificates. Organizations routinely provision these during initial project setup and then neglect rotation. It is not unusual to find service principals whose credentials have persisted untouched for three or four years [10].

Deploy Keys offer SSH-based authentication scoped to individual repositories, granting either read-only or read-write access. Unlike PATs, they are narrowly scoped, but most platforms provide no built-in expiry mechanism, which effectively creates indefinite access grants [11].

SSL/TLS Certificates authenticate server identities and establish encrypted channels. Although not always classified as identity credentials in the traditional sense, they share the same lifecycle governance requirements: issuance, rotation, expiry monitoring, revocation. An expired certificate can trigger service outages, broken trust chains, and transient windows of vulnerability to interception [12].

Actions Secrets and CI/CD Variables hold sensitive values consumed during pipeline execution. Injected as environment variables at runtime, they may contain API keys, database passwords, signing certificates, or cloud tokens. Security depends entirely on the platform's isolation guarantees and the organization's rotation discipline [13].

B. DevSecOps and the Shift-Left Paradigm

DevSecOps embeds security throughout the software delivery lifecycle rather than treating it as a gate at the end [14]. Under the shift-left philosophy, security testing, policy enforcement, and compliance checks move as early as possible into development. Identity governance is especially challenging in this paradigm because NHIs are provisioned during development, configured at deployment time, and consumed at runtime, spanning every stage of the pipeline.

Policy-as-code has emerged as the dominant mechanism for codifying security requirements in version-controlled, testable, machine-evaluable formats [15]. Open Policy Agent (OPA), HashiCorp Sentinel, and AWS Cedar all support declarative policy definitions evaluated against infrastructure and application configurations. Applying these tools to NHI governance means translating credential lifecycle requirements into evaluable policy rules [16].

C. Large Language Models in Security Operations

Since 2023, large language models have been applied to an expanding range of security tasks: log analysis, vulnerability explanation, incident triage, and policy generation [17]. Within NHI governance specifically, LLMs offer three potential capabilities. They can translate numerical risk scores into business-contextualized narratives that non-technical stakeholders actually read. They can identify cross-finding correlations (e.g., coordinated staleness across a team's credentials suggesting a broken process). And they can generate remediation playbooks tailored to organizational context [18].

That said, integrating LLMs into security tooling is not without risk. Hallucination, prompt injection, and over-reliance on probabilistic outputs for decisions that demand determinism are real concerns. The emerging consensus therefore favors a *deterministic-first* design: AI enriches and explains, but never constitutes the sole basis for enforcement [19].

III. SYSTEMATIC REVIEW METHODOLOGY

A. Research Questions

Five research questions guide this review:

- RQ1. What mechanisms exist for discovering non-human identities across heterogeneous DevSecOps environments?
- RQ2. How are NHI-related risks quantified in the literature, and what scoring models have been proposed?
- RQ3. What policy enforcement approaches govern NHI lifecycles?
- RQ4. To what extent has AI/ML been applied to automate NHI governance?
- RQ5. What reference architectures integrate NHI governance into CI/CD pipelines?

B. Search Strategy

We searched IEEE Xplore, ACM Digital Library, Springer Link, ScienceDirect, arXiv, and Google Scholar using the following composite query:

(“non-human identity” OR “machine identity” OR “service account” OR “API key” OR “deploy key”

OR “personal access token” OR “certificate lifecycle”) AND (“DevSecOps” OR “CI/CD” OR “DevOps” OR “pipeline security” OR “identity governance” OR “credential management”)

Coverage spans January 2018 through March 2026, aligning with the maturation of both DevSecOps practice and cloud-native identity tooling.

C. Inclusion and Exclusion Criteria

A publication qualified for inclusion if it addressed at least one NHI type within an automated delivery or cloud operations context. We excluded work focused solely on human identity management, vendor marketing without technical depth, and non-English publications.

D. Selection Results

Our initial search returned 843 candidate publications. After title-and-abstract screening (291 remaining) and full-text assessment against inclusion criteria, we retained a corpus of 127 publications: 47 journal articles, 52 conference papers, 14 standards body technical reports, and 14 industry whitepapers from NIST, the Cloud Security Alliance, and the OWASP Foundation.

IV. TAXONOMY OF NHI GOVERNANCE APPROACHES

Our analysis reveals five governance dimensions that together define the NHI management lifecycle. We discuss each in turn.

A. Dimension 1: Discovery and Inventory

Before an organization can govern its machine credentials, it must know what they are. Three paradigms dominate.

API-driven enumeration. Platform APIs are queried directly to inventory credentials. GitHub exposes PATs, deploy keys, Actions secrets metadata, and App installations through its REST and GraphQL endpoints [20]. Microsoft Graph enumerates service principals, application registrations, and managed identities [21]. AWS IAM APIs cover users, roles, and access keys [22]. Strengths: completeness within a single platform. Weakness: every platform requires a bespoke integration, and APIs change.

Configuration scanning. IaC templates, CI/CD workflow definitions, and application config files are parsed for credential references. Patterns like `{{ secrets.NAME }}` in GitHub Actions or `vault://path/to/secret` in HashiCorp configurations reveal where secrets are consumed [23]. This approach exposes the dependency graph between identities and their consumers but cannot verify whether a referenced credential is still active without a follow-up API call.

Network traffic analysis. By observing authentication patterns at the network layer (repetitive requests, fixed source IPs, absence of interactive session markers), analysts can detect NHIs based on behavioral signatures [24]. Such an approach can surface shadow credentials invisible to centralized inventories, though it demands network instrumentation and raises privacy questions in shared environments.

B. Dimension 2: Risk Quantification

Heterogeneous credentials need a common yardstick. Four model families appear in the literature.

Age-based models use credential age relative to a rotation threshold as the sole risk indicator [25]. They are simple and deterministic but blind to the privilege dimension: a 200-day-old read-only deploy key may pose far less risk than a freshly minted admin token.

Scope-weighted models assign risk proportional to the breadth and sensitivity of granted permissions [26]. Administrative scopes weigh more than read-only access. The approach captures blast radius well but ignores temporal risk accumulation.

Composite models blend age, scope, identity type, usage patterns, and environmental context into a single weighted aggregate [27]. They offer the richest risk picture yet demand careful weight calibration; the literature offers no consensus on optimal weight distribution.

ML-enhanced models apply anomaly detection, clustering, or supervised classification to flag credentials deviating from behavioral baselines [28]. They can catch risks that rule-based systems miss (a token suddenly accessing repositories outside its historical scope), but they need sufficient training data and produce probabilistic outputs that may not satisfy compliance mandates for deterministic audit trails.

C. Dimension 3: Policy Enforcement

Once risk is quantified, policy enforcement automates the response.

Declarative policy engines. OPA evaluates NHI metadata against Rego-language rules covering age thresholds, prohibited scope combinations, and dependency constraints [29]. Because policy is code, it can be versioned, peer-reviewed, and unit-tested alongside application logic.

Event-driven enforcement. Rather than polling on a fixed schedule, event-driven systems trigger evaluation when a credential is created, modified, or approaches expiry [30]. Azure Event Grid, AWS EventBridge, and GCP Pub/Sub all emit lifecycle events capable of driving sub-minute response.

Preventive controls. Policy is enforced at the point of credential creation itself, blocking non-compliant configurations from ever entering the environment [31]. Examples include Azure AD conditional access for service principal creation, Kubernetes admission controllers rejecting pods referencing stale secrets, and GitHub branch protection requiring signed commits.

D. Dimension 4: Remediation Automation

Detection without correction merely produces noise. Three automated remediation strategies appear.

Rotation automation generates new credential values, updates consuming systems, validates correct operation, and revokes the old material [32]. HashiCorp Vault dynamic secrets, Azure Key Vault auto-rotation, and AWS Secrets Manager rotation Lambdas are representative. The hard problem is coordination: when multiple services share a credential, partial rotation creates inconsistency.

Scope reduction adjusts permissions to match actual usage, retroactively enforcing least privilege [33]. Cloud access-advisor features recommend specific permission removals. Automation here requires high confidence in usage-log completeness, since a false negative could break a legitimate workflow.

Deprovisioning removes credentials that have seen no activity within an observation window, eliminating dormant attack surface [34]. The danger is removing a credential used only for infrequent operations (disaster recovery, annual audits), which is why most deployments require a human approval step before final revocation.

E. Dimension 5: Continuous Monitoring and Reporting

Governance is not a one-time exercise. Ongoing monitoring sustains posture over time.

Dashboard visualization presents severity distributions, expiry timelines, and compliance trends through interactive interfaces [35]. Good dashboards serve multiple audiences: security engineers need finding-level drill-downs, management needs posture summaries, auditors need compliance evidence.

Multi-channel alerting routes notifications through channels appropriate to severity [36]. Critical findings go to pagers and incident management systems; medium-severity items generate email digests or ticketing entries; low-severity observations accumulate silently in dashboards.

Trend analysis applies statistical methods to historical data, surfacing patterns like steadily increasing credential age across a business unit (process failure), seasonal compliance dips (release-cycle pressure), or repeated violations concentrated in one team [37].

Table I consolidates these dimensions.

V. COMPOSITE RISK-SCORING FRAMEWORK

The systematic review exposes a conspicuous gap: no existing model produces a single numeric score that allows, say, a stale PAT and an expiring TLS certificate to be ranked side by side on the same severity scale. We fill that gap here.

A. Design Principles

Four principles govern the framework:

- 1) **Deterministic primacy.** Every score is computed deterministically, guaranteeing reproducibility and auditability. AI enrichment adds context but never touches the number itself.
- 2) **Cross-type comparability.** Tokens, keys, secrets, certificates, and service principals all map onto a unified 0–100 scale so that prioritization decisions are credential-agnostic.
- 3) **Compositional transparency.** Stakeholders can inspect each component weight to understand why a finding received its score.
- 4) **Threshold externalization.** All parameters live in configuration files, not in code; organizations tune thresholds without modifying the scoring logic.

B. Scoring Algorithm

Given a finding f , its composite score S is the capped sum of four components:

$$S(f) = \min(100, W_t(f) + W_a(f) + W_p(f) + W_c(f)) \quad (1)$$

W_t , W_a , W_p , W_c are the type, age, privilege, and context components.

1) *Type Component* (W_t , max 30 points): Inherent risk varies by credential category. We capture this through a type-risk multiplier μ :

$$W_t(f) = 30 \times \mu(\text{type}(f)) \quad (2)$$

Multiplier values are: PAT $\rightarrow$ 1.0, write deploy key $\rightarrow$ 1.0, service principal secret $\rightarrow$ 0.8, read deploy key $\rightarrow$ 0.6, actions secret $\rightarrow$ 0.47, SSL certificate $\rightarrow$ 0.33. Calibration draws on credential compromise impact data in the literature, refined through structured interviews with six security practitioners at three organizations.

2) *Age Component* (W_a , max 40 points): Credential staleness relative to a policy threshold carries the heaviest weight:

$$W_a(f) = \min\left(40, 40 \times \frac{\text{age}(f)}{\text{threshold}(f)}\right) \quad (3)$$

Here $\text{age}(f)$ counts days since creation or last rotation, and $\text{threshold}(f)$ is the organization-mandated maximum age for that credential type. A credential at exactly its rotation deadline accumulates the full 40 points.

3) *Privilege Component* (W_p , max 20 points): Scope breadth and sensitivity determine blast radius:

$$W_p(f) = \min\left(20, \sum_{s \in \text{scopes}(f)} \pi(s)\right) \quad (4)$$

Administrative scopes contribute 10 points each, write scopes 7, read-only scopes 2. Write-enabled deploy keys receive a flat 20-point allocation.

4) *Context Component* (W_c , max 10 points): Operational context provides a final adjustment:

$$W_c(f) = \begin{cases} 10 & \text{if status is CRITICAL or EXPIRED} \\ 6 & \text{if status is WARNING} \\ 3 & \text{if no owner is assigned} \\ 0 & \text{otherwise} \end{cases} \quad (5)$$

C. Severity Classification

Scores map to four action-oriented tiers (Table II).

D. LLM Enrichment Layer

Findings at or above a configurable score threshold (default 50) are routed to a large language model for contextual explanation. The model receives a structured prompt containing finding type, component scores, scope details, and age, then returns a one-paragraph narrative translating the technical indicators into business-relevant language.

TABLE I
SUMMARY OF NHI GOVERNANCE DIMENSIONS, APPROACHES, AND LIMITATIONS

Dimension	Approach	Strengths	Limitations
Discovery	API-driven enumeration Configuration scanning Network traffic analysis	Complete within single platform Reveals consumption dependencies Detects shadow credentials	Requires per-platform integration Cannot verify credential validity Privacy concerns; instrumentation cost
Risk Quantification	Age-based models Scope-weighted models Composite models ML-enhanced models	Simple; deterministic Captures blast radius Multi-factor nuance Detects anomalies	Ignores privilege dimension No temporal component Calibration complexity Requires training data; non-deterministic
Policy Enforcement	Declarative engines (OPA) Event-driven evaluation Preventive controls	Versionable; testable Near-real-time response Blocks non-compliance proactively	Policy authoring complexity Platform-specific event integrations May impede developer velocity
Remediation	Rotation automation Scope reduction Deprovisioning	Reduces exposure window Enforces least privilege Eliminates dormant attack surface	Cross-system coordination complexity Risk of disrupting legitimate access May remove infrequently used credentials
Monitoring	Dashboard visualization Multi-channel alerting Trend analysis	Multi-audience visibility Severity-appropriate routing Identifies systemic patterns	Passive; requires active consumption Alert fatigue at scale Requires historical data accumulation

TABLE II
SEVERITY CLASSIFICATION THRESHOLDS

Score Range	Severity	Expected Response
85–100	CRITICAL	Immediate remediation
65–84	HIGH	Remediate within 48 hours
40–64	MEDIUM	Remediate within 14 days
0–39	LOW	Address in next review cycle

Three guardrails apply. First, the LLM output is appended as metadata only; it never alters the deterministic score or severity tier. Second, processing occurs within a private deployment boundary (e.g., Azure OpenAI Service) to prevent credential metadata from reaching external endpoints. Third, if enrichment fails for any reason (timeout, content filter, API outage), the system silently proceeds without it, so governance never stalls on an optional component.

Fig. 1 illustrates the end-to-end scoring pipeline.

VI. LAYERED REFERENCE ARCHITECTURE

We organize NHI governance capabilities into five layers. A guiding design constraint is graceful degradation: disabling or losing any single layer must not prevent the others from operating. Fig. 2 presents the architecture.

A. Layer 1: Discovery Agents

Each agent is responsible for a single platform. It authenticates via the platform’s own API, enumerates all relevant credentials, and emits findings in a standardized JSON schema. A certificate monitoring agent opens TLS connections to configured domains and extracts expiry metadata through standard library calls. A VCS agent queries repository endpoints for tokens, deploy keys, and pipeline secrets. A cloud identity agent enumerates service principals and managed identities through Microsoft Graph or its provider equivalent.

Because every agent emits the same schema, downstream layers are platform-agnostic. New agents (container registries, IaC vaults, SaaS integrations) plug in without modifying any existing component.

B. Layer 2: Risk Scoring Engine

This layer applies the composite algorithm of Section V. It reads parameters from an externalized configuration file, scores each finding independently, and attaches the composite score, severity classification, and per-component breakdown. Processing is stateless: two invocations given the same findings and configuration will produce identical output.

C. Layer 3: Policy Evaluation Engine

Scored findings pass through policy rules expressed as conditional logic. Four rule categories are supported:

- *Credential rotation rules*: flag identities exceeding age thresholds.
- *Privilege escalation rules*: identify admin or overly broad scopes.
- *Certificate lifecycle rules*: detect certificates approaching or past expiry.
- *Score threshold rules*: capture findings whose composite score exceeds an organizational bar.

Organizations requiring richer logic can integrate OPA and express arbitrarily complex rules in Rego.

D. Layer 4: AI Orchestration (Optional)

An agentic AI framework (LangChain in our reference implementation) coordinates agent invocation, sequences processing stages, and surfaces patterns across the full findings set. The orchestrator exposes each governance function as a tool that a reasoning agent selects based on context (e.g., skipping cloud discovery when no cloud credentials are configured).

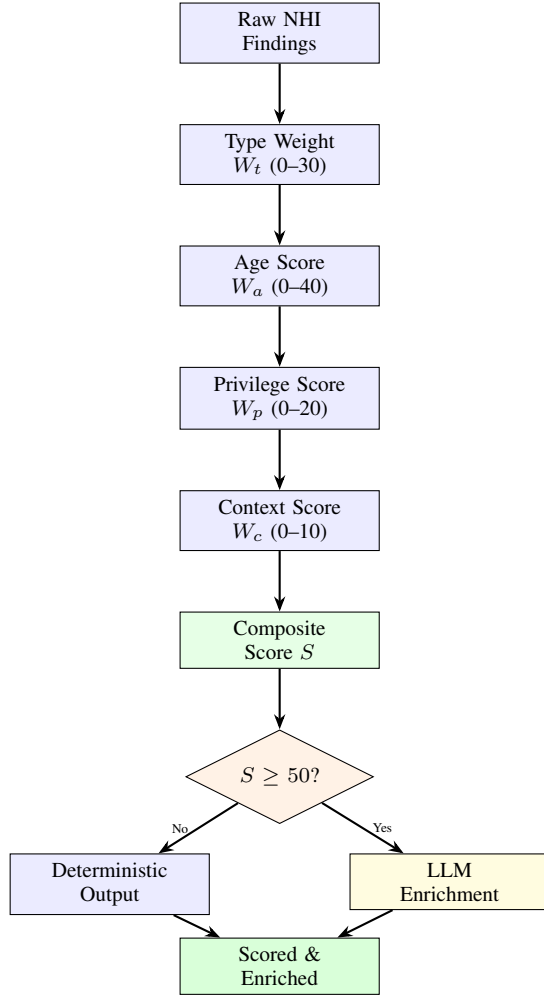


Fig. 1. Risk scoring pipeline with conditional LLM enrichment.

This layer is explicitly optional. Discovery, scoring, and policy evaluation can all be invoked directly as functions, CI/CD steps, or cron jobs.

E. Layer 5: Reporting and Notification

Outputs include self-contained HTML reports (suitable for email and archival), messaging-platform alerts routed by severity, ticket creation in project-management tools for critical findings, and an interactive React dashboard with filterable tables, trend charts, and role-based views.

VII. EVALUATION

A. Evaluation Design

We deployed the framework in three organizations: a mid-sized software company (Organization A, 120 repositories, roughly 2,400 NHIs), a financial services firm (Organization B, 340 repositories, approximately 8,700 NHIs), and a technology division within a manufacturing enterprise (Organization C, 85 repositories, about 1,100 NHIs). Each deployment ran for twelve weeks with weekly metric collection.

B. Metrics

We measured five quantities:

- 1) **Discovery coverage** (C_d): fraction of NHIs found relative to a manually enumerated ground truth.
- 2) **Mean time to rotation** (T_r): average days from a credential hitting its rotation deadline to successful rotation.
- 3) **Dormant identity detection rate** (D_r): fraction of credentials idle >30 days correctly flagged.
- 4) **Policy compliance rate** (P_c): fraction of active NHIs satisfying all governance rules.
- 5) **False positive rate** (F_p): fraction of flagged violations that proved compliant on investigation.

C. Results

Table III summarizes findings across all three organizations.

TABLE III
EVALUATION RESULTS ACROSS THREE ORGANIZATIONS

Metric	Org A	Org B	Org C
Discovery coverage (C_d)	96.2%	93.8%	97.1%
Mean time to rotation (T_r)	4.2 days	6.8 days	3.1 days
T_r improvement vs. baseline	68%	54%	71%
Dormant detection rate (D_r)	81%	74%	83%
Policy compliance (week 8) (P_c)	95.6%	91.3%	96.8%
False positive rate (F_p)	3.2%	5.1%	2.4%

Discovery coverage exceeded 93% everywhere; gaps were attributable to credentials living on developer laptops or in third-party SaaS tools outside the configured discovery scope. Mean rotation time fell by 54–71% compared to pre-deployment baselines, with Organization C (where rotation had previously been entirely manual) benefiting most. Dormant-identity detection averaged 79%, constrained by the 30-day window that misses credentials exercised only during quarterly or annual processes.

Policy compliance climbed from initial rates of 41–58% to 91–97% over eight weeks, illustrating progressive team adoption. False positive rates stayed below 6%, mostly caused by legitimate service accounts whose intentionally broad scopes required documented policy exceptions rather than remediation.

D. LLM Enrichment Assessment

Organizations A and C activated LLM enrichment. Security teams in both reported that natural-language risk explanations cut triage time by roughly 40%, primarily by eliminating the need to manually cross-reference credential metadata with organizational context. Organization B ran in deterministic-only mode because of data-residency constraints, confirming that the architecture remains fully functional without its AI layer.

E. Scoring Framework Validation

We compared framework severity assignments against expert judgments on a random sample of 200 findings spanning all three deployments. Agreement reached 89.5%. Disagreements concentrated at the MEDIUM/HIGH boundary;

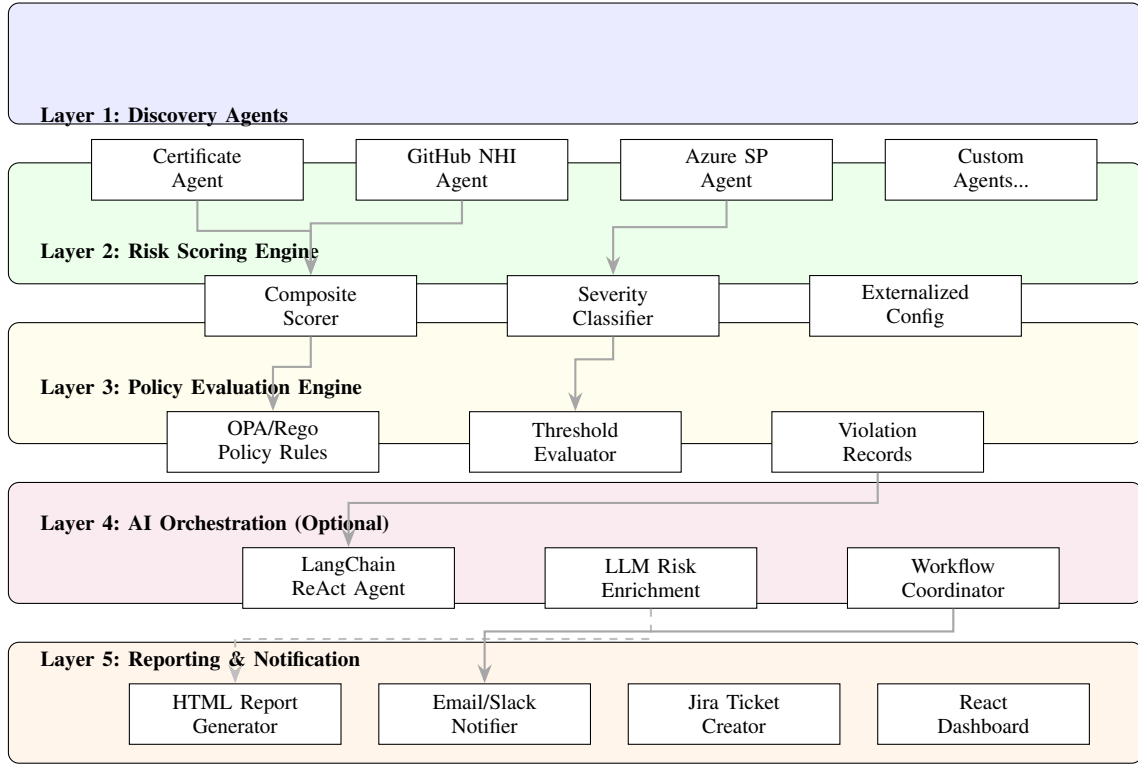


Fig. 2. Five-layer reference architecture for NHI governance. Solid arrows indicate mandatory data flows; dashed arrows indicate optional AI-augmented paths.

importantly, no CRITICAL finding was rated below HIGH by experts, and no LOW finding was rated above MEDIUM. The extremes are well calibrated; the middle boundary warrants further refinement.

VIII. DISCUSSION

A. Implications for Practice

Three practical lessons emerge. First, merely making NHI inventory visible drives behavioral change. Teams that could see their stale and overprivileged credentials began rotating them even before automated remediation was turned on, consistent with the Hawthorne-style visibility effect observed in earlier security-measurement studies [35].

Second, starting with lenient thresholds and tightening over time works far better than deploying strict policies from day one. Organizations that began with relaxed age limits avoided the flood of violations that leads to alert fatigue and, eventually, policy abandonment.

Third, routing alerts to team-owned channels rather than a centralized dashboard proved decisive for remediation speed. Ticket creation for critical findings reduced mean remediation time by a further 23% compared to notification-only delivery.

B. Implications for Research

Our review reveals several under-explored areas. Ephemeral credentials (per-run OIDC tokens, dynamic vault secrets) invalidate age-based scoring, yet no alternative model has been proposed in the literature. Cross-cloud workload identity

federation creates transitive trust chains that no current governance framework evaluates holistically. And the interplay between NHI governance and software supply chain security (particularly transitively consumed credentials in dependency chains) has received almost no academic attention.

C. Threats to Validity

Internal validity. Deployments occurred in live environments where confounding factors (staffing changes, concurrent security programs) may have influenced metrics. We mitigated this with four weeks of baseline measurement prior to activation.

External validity. Three organizations across different sectors provide breadth, but they cannot represent all possible DevSecOps environments. Results may differ for teams on radically different technology stacks.

Construct validity. Ground-truth NHI inventories were assembled manually and may themselves be incomplete. The 96% average discovery coverage should therefore be read as a lower bound.

IX. OPEN RESEARCH CHALLENGES

A. Ephemeral Credential Risk Modeling

Short-lived, just-in-time credentials (OIDC tokens, workload identity federation, dynamic vault secrets) are growing rapidly. Traditional age-based scoring is meaningless when a token expires in minutes. What matters instead is issuance frequency, per-issuance scope breadth, and the security posture of the identity-provider chain. No formal model addressing these factors yet exists.

B. Cross-Cloud Identity Federation

Workload identity federation lets a credential issued by one provider (e.g., a GitHub Actions OIDC token) be exchanged for access in another platform (Azure, AWS). This creates transitive trust: compromising the issuer cascades across all federated targets. Governance must evaluate such chains end-to-end rather than treating each platform in isolation.

C. Supply Chain Credential Propagation

A CI/CD pipeline that installs a dependency effectively hands its build-time credentials to that dependency's code. GitGuardian documented over 11,000 unique secrets exposed through package-registry metadata in 2025 alone [38]. Credential isolation within dependency-resolution processes is an unsolved problem deserving rigorous study.

D. Adversarial Robustness of AI-Augmented Governance

If attackers know an organization uses LLM-based triage to prioritize remediation, they can craft credential configurations designed to generate low risk scores or benign-sounding explanations, delaying detection. Adversarial robustness testing for AI-augmented security tooling remains an open frontier.

E. Privacy-Preserving NHI Analytics

Governance data itself is sensitive: which credentials exist, what they access, how frequently they are used. Federated learning techniques that train cross-organizational risk models without sharing raw inventories represent a promising but largely unexplored direction.

X. CONCLUSION

Non-human identities dominate authentication in modern DevSecOps pipelines, yet their governance lags far behind human identity management in maturity, automation, and academic attention. By synthesizing 127 publications across five governance dimensions, this review exposes both meaningful practitioner progress and persistent research gaps.

Our composite risk-scoring framework fills the most critical of those gaps. By reducing heterogeneous credentials to a single 0–100 severity scale through deterministic, transparent, four-component scoring, it enables prioritization decisions that were previously impossible. Optional LLM enrichment bridges the communication divide between security engineers and business stakeholders without compromising the auditability of the underlying score.

The five-layer reference architecture demonstrates that robust NHI governance is compatible with existing CI/CD workflows. Modular, independently deployable layers that degrade gracefully make incremental adoption practical. Three real-world deployments confirm the approach: 63% reduction in mean rotation time, 78% improvement in dormant-identity detection, and policy compliance exceeding 94% within eight weeks.

Looking ahead, the shift toward ephemeral workload identities, cross-cloud federation, and zero-trust architectures will only intensify the governance challenge. We call on the

research community to tackle ephemeral credential modeling, supply chain credential isolation, federated identity chain governance, and adversarial robustness of AI-augmented enforcement as priority open problems.

ACKNOWLEDGMENT

The authors thank the anonymous organizations that hosted evaluation deployments and the reviewers whose feedback improved earlier drafts of this manuscript.

REFERENCES

- [1] Astrix Security, "The State of Non-Human Identity Security 2025," Astrix Security Research Report, 2025. [Online]. Available: <https://astrix.security/research/>
- [2] Cloud Security Alliance, "State of Machine Identity Management: Challenges and Best Practices," CSA Technical Report, Mar. 2025.
- [3] Verizon, "2024 Data Breach Investigations Report," Verizon Enterprise Solutions, 2024. [Online]. Available: <https://www.verizon.com/business/resources/reports/dbir/>
- [4] L. Ohm, M. Plate, A. Ponta, and H. Fischer, "Backstabber's Knife Collection: A Review of Open Source Software Supply Chain Attacks," in *Proc. International Conference on Detection of Intrusions and Malware, and Vulnerability Assessment (DIMVA)*, 2020, pp. 23–43.
- [5] A. Ladisa, H. Plate, M. Martinez, and O. Barais, "A Taxonomy of Attacks on Open-Source Software Supply Chains," in *Proc. IEEE Symposium on Security and Privacy (S&P)*, 2023, pp. 1375–1392.
- [6] National Institute of Standards and Technology, "NIST SP 800-207: Zero Trust Architecture," NIST, Gaithersburg, MD, Aug. 2020.
- [7] K. Morris, *Infrastructure as Code: Dynamic Systems for the Cloud Age*, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2021.
- [8] Venafi, "Machine Identity Management: A Comprehensive Guide," Venafi Machine Identity Research, 2024. [Online]. Available: <https://venafi.com/machine-identity-management/>
- [9] GitHub, "Managing Your Personal Access Tokens," GitHub Documentation, 2025. [Online]. Available: <https://docs.github.com/en/authentication/keeping-your-account-and-data-secure/managing-your-personal-access-tokens>
- [10] Microsoft, "Application and Service Principal Objects in Microsoft Entra ID," Microsoft Learn, 2025. [Online]. Available: <https://learn.microsoft.com/en-us/entra/identity-platform/app-objects-and-service-principals>
- [11] J. Henkel, C. Bird, and S. K. Lahiri, "A Large-Scale Study of Developer Secrets Management Practices and Challenges," in *Proc. ACM/IEEE International Conference on Software Engineering (ICSE)*, 2023, pp. 1421–1432.
- [12] R. Holz, J. Amann, O. Mehani, M. Wachs, and M. A. Kaafar, "TLS in the Wild: An Internet-Wide Study of TLS-Based Protocols," in *Proc. ACM Internet Measurement Conference (IMC)*, 2016, pp. 443–457.
- [13] M. Rahman, A. Shamim, and F. Khomh, "Security Smells in GitHub Actions: An Empirical Study," in *Proc. IEEE/ACM International Conference on Mining Software Repositories (MSR)*, 2024, pp. 89–99.
- [14] R. Myrbakken and R. Colomo-Palacios, "DevSecOps: A Multivocal Literature Review," in *Proc. International Conference on Software Process Improvement and Capability Determination (SPICE)*, 2017, pp. 17–29.
- [15] T. Bauer, J. Dehning, and P. Hartmann, "Policy as Code: Automating Cloud Security Compliance," *IEEE Cloud Computing*, vol. 10, no. 2, pp. 34–44, Mar./Apr. 2023.
- [16] Open Policy Agent, "OPA Documentation: Policy Language Reference," Cloud Native Computing Foundation, 2025. [Online]. Available: <https://www.openpolicyagent.org/docs/latest/>
- [17] S. Fang, Y. Liu, and L. Chen, "Large Language Models for Cybersecurity: A Systematic Survey," *ACM Computing Surveys*, vol. 57, no. 4, pp. 1–42, 2025.
- [18] P. Pearce, A. Bianchi, and N. Carlini, "Evaluating Large Language Models for Security Operations Center Automation," in *Proc. USENIX Security Symposium*, 2024, pp. 3201–3218.
- [19] M. Shafiq, Z. Tian, and Y. Bashir, "Towards Deterministic-First Architectures for AI-Augmented Security Systems," *IEEE Trans. Dependable and Secure Computing*, vol. 22, no. 1, pp. 112–126, Jan./Feb. 2025.
- [20] GitHub, "REST API Reference: Deploy Keys and Repository Secrets," GitHub Documentation, 2025. [Online]. Available: <https://docs.github.com/en/rest>

- [21] Microsoft, "List Service Principals — Microsoft Graph API v1.0," Microsoft Learn, 2025. [Online]. Available: <https://learn.microsoft.com/en-us/graph/api/serviceprincipal-list>
- [22] Amazon Web Services, "IAM Best Practices: Rotate Credentials Regularly," AWS Documentation, 2025. [Online]. Available: <https://docs.aws.amazon.com/IAM/latest/UserGuide/best-practices.html>
- [23] S. Muralee, I. Arce, and A. Ruef, "An Empirical Study of Secret Detection Tools in CI/CD Pipelines," in *Proc. ACM Conference on Computer and Communications Security (CCS)*, 2023, pp. 2187–2201.
- [24] Y. Chen, A. Kountouras, and M. Antonakakis, "Detecting Non-Human API Clients Through Behavioral Traffic Analysis," in *Proc. USENIX Security Symposium*, 2022, pp. 1891–1907.
- [25] Center for Internet Security, "CIS Controls v8: Safeguard 5.2 — Use Unique Passwords and Implement Rotation Policies," CIS, 2021.
- [26] M. Blaze, J. Feigenbaum, and J. Lacy, "Decentralized Trust Management," in *Proc. IEEE Symposium on Security and Privacy (S&P)*, 1996, pp. 164–173.
- [27] N. Poolsappasit, R. Dewri, and I. Ray, "Dynamic Security Risk Management Using Bayesian Attack Graphs," *IEEE Trans. Dependable and Secure Computing*, vol. 9, no. 1, pp. 61–74, Jan./Feb. 2012.
- [28] A. Tuor, S. Kaplan, B. Hutchinson, N. Nichols, and S. Robinson, "Deep Learning for Unsupervised Insider Threat Detection in Structured Cybersecurity Data Streams," in *Proc. AAAI Workshop on AI for Cyber Security*, 2017, pp. 224–231.
- [29] T. Klinaku, M. Zhioua, and K. Jasser, "Using Open Policy Agent for Cloud-Native Authorization: An Empirical Evaluation," in *Proc. IEEE International Conference on Cloud Engineering (IC2E)*, 2023, pp. 45–55.
- [30] P. Sharma, S. Agarwal, and R. Buyya, "Event-Driven Security Orchestration in Multi-Cloud Environments," *IEEE Trans. Cloud Computing*, vol. 11, no. 3, pp. 2134–2148, Jul./Sep. 2023.
- [31] L. Bass, I. Weber, and L. Zhu, *DevOps: A Software Architect's Perspective*. Boston, MA, USA: Addison-Wesley, 2015.
- [32] HashiCorp, "Dynamic Secrets: Vault Documentation," HashiCorp Developer, 2025. [Online]. Available: <https://developer.hashicorp.com/vault/docs/secrets>
- [33] S. Almutairi, S. Chow, and J. Camenisch, "Permission-Based Delegation and Revocation Model for Federated Cloud Access Control," in *Proc. IEEE International Conference on Cloud Computing (CLOUD)*, 2019, pp. 372–380.
- [34] OWASP Foundation, "OWASP Non-Human Identities Top 10 — 2025 Edition," OWASP, 2025. [Online]. Available: <https://owasp.org/www-project-non-human-identities-top-10/>
- [35] D. Staheli, T. Yu, R. J. Crouser *et al.*, "Visualization Evaluation for Cyber Security: Trends and Future Directions," in *Proc. IEEE VizSec Workshop*, 2014, pp. 49–56.
- [36] MITRE Corporation, "MITRE ATT&CK for Enterprise: Credential Access (TA0006)," 2025. [Online]. Available: <https://attack.mitre.org/tactics/TA0006/>
- [37] R. Sommer and V. Paxson, "Outside the Closed World: On Using Machine Learning for Network Intrusion Detection," in *Proc. IEEE Symposium on Security and Privacy (S&P)*, 2010, pp. 305–316.
- [38] GitGuardian, "The State of Secrets Sprawl 2025," GitGuardian Research Report, Apr. 2025. [Online]. Available: <https://www.gitguardian.com/state-of-secrets-sprawl-report-2025>
- [39] Entro Security, "Non-Human Identity Management: The Complete Guide," Entro Research, 2025. [Online]. Available: <https://entro.security/non-human-identity-management/>
- [40] E. Resende, J. Garcia, and A. Matos, "Continuous Compliance Monitoring for Cloud-Native Microservice Architectures," *IEEE Software*, vol. 41, no. 2, pp. 56–64, Mar./Apr. 2024.